

Lecture Note: Functions In Python



Nishut Suman

10 Jan, 2026 At 11:00 AM

Notes

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Lecture Note: Functions in Python

Functions in Python are defined using the **def** keyword, followed by the function name, parentheses () which may contain parameters, and a colon :. The function's body is then indented on subsequent lines.

Prerequisites

Before learning about functions, make sure you are comfortable with:

- Basic Python syntax
- Variables and data types
- Indentation and code blocks
- Printing and input statements

Why We Use Functions in Python

Functions make code cleaner, reusable, and easier to manage.

- **Reusability** – Write once, use many times; saves effort and follows the DRY principle.
- **Modularity** – Breaks large code into smaller, manageable parts.
- **Readability** – Clear function names make code easier to understand.
- **Maintainability** – Easy to update or fix specific parts without affecting the whole program.
- **Abstraction** – Focus on what a function does, not how it works.
- **Collaboration** – Enables multiple developers to work on different parts simultaneously.

Function Syntax

```
def function_name(parameter1, parameter2):  
    # Function body starts here, indented  
    # Perform operations using parameter1 and parameter2  
    result = parameter1 + parameter2  
    return result # Optional: return a value
```

- **def:** This keyword signals the start of a function definition.
- **function_name:** Identifier for the function. Should follow Python's naming conventions (lowercase with underscores).
- **():** Enclose the function's parameters (optional). Parameters are placeholders for values passed into the function.
- **:" (Colon)** - Marks the end of the function header and beginning of the function body.
- **Indentation:** Code inside a function must be indented (typically 4 spaces).
- **return:** Optional statement used to send a value back to the caller.

Example: Defining a Function with No Parameters

This function simply prints a message.

```
def greet():  
    print("Hello, welcome to Python!")  
  
# The function is defined but does not run until called.
```

Calling a Function in Python

To call a function, type its name followed by parentheses.

If the function accepts arguments, pass them inside the parentheses.

1. Basic Function Call

Call the function by its name followed by parentheses.

```
def greet():  
    print("Hello, world!")  
  
greet() # Output: Hello, world!
```

2. Function with Arguments

Pass the required arguments inside the parentheses.

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(3, 5)  
print(result) # Output: 8
```

Parameters vs Arguments in Python

In Python, **functions** often require input values to perform operations.

These inputs are known as **parameters** and **arguments**, but they are **not the same thing**.

Difference Between Parameters and Arguments

Term	Meaning	Example
Parameters	Variables defined in the function definition that accept values when the function is called.	<code>def greet(name):</code> → here, <code>name</code> is a parameter
Arguments	Actual values passed to the function when it is called.	<code>greet("Rahul")</code> → <code>"Rahul"</code> is an argument

Think of it this way:

- Parameters = placeholders
- Arguments = actual data passed to those placeholders

Types of Arguments in Python

Python supports several types of arguments:

Positional Arguments

Keyword Arguments

Default Arguments

1. Positional Arguments

The **order** in which you pass arguments **matters** — each value is assigned in the same order.

```
def student_info(name, age, city):  
    print(f"Name: {name}, Age: {age}, City: {city}")  
  
# Positional arguments  
student_info("Rahul", 24, "Delhi")
```

Output:

```
Name: Rahul, Age: 24, City: Delhi
```

If you change the order:

```
student_info(24, "Rahul", "Delhi")
```

Output:

```
Name: 24, Age: Rahul, City: Delhi
```

2. Keyword Arguments

You can pass arguments by **specifying parameter names**, regardless of order.

```
def student_info(name, age, city):  
    print(f"Name: {name}, Age: {age}, City: {city}")  
  
student_info(age=24, name="Rahul", city="Delhi")
```

Output:

```
Name: Rahul, Age: 24, City: Delhi
```

The order doesn't matter because parameters are explicitly named.

3. Default Arguments

Assign **default values** to parameters in the function definition.

If no argument is provided, the **default value** is used.

```
def greet(name="Guest"):  
    print(f"Hello, {name}!")  
  
greet("Rahul")  
greet()
```

Output:

```
Hello, Rahul!  
Hello, Guest!
```

Default arguments make functions flexible and prevent missing argument errors.

Combination of Arguments

You can combine different argument types, but the order matters:

Positional → Default → Variable → Keyword

Example:

```
def display_info(name, age=25, city="Delhi"):  
    print(f"Name: {name}, Age: {age}, City: {city}")  
  
display_info("Rahul")  
display_info("Priya", city="Mumbai")
```

Output:

```
Name: Rahul, Age: 25, City: Delhi  
Name: Priya, Age: 25, City: Mumbai
```

Quick Summary Table

Type	Description	Example
Positional	Arguments passed in order	<code>func(10, 20)</code>
Keyword	Arguments passed by name	<code>func(a=10, b=20)</code>
Default	Predefined values if not passed	<code>def func(a=10)</code>

Key Takeaways

- **Parameters** → defined in function
- **Arguments** → values passed when calling
- **Positional** → order matters
- **Keyword** → order doesn't matter
- **Default** → uses predefined values if not supplied

Try It Yourself

```
def order(item, quantity=1, price=100):  
    print(f"Item: {item}, Quantity: {quantity}, Total: {quantity * price}")  
  
order("Book")  
order("Pen", 5)  
order(price=50, item="Bag")
```

Return Values and Variable Scope in Python

1. Return Values in Python

A **return statement** sends a value back to where the function was called, allowing it to **produce a result** that can be stored or used.

Using return

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(5, 3)
```

```
print(result)
```

Output:

```
8
```

Function Without Return

If no return is used, Python automatically returns **None**.

```
def greet():  
    print("Hello, Python!")  
  
value = greet()  
print(value)
```

Output:

```
Hello, Python!  
None
```

Returning Multiple Values

You can return more than one value using **tuples**.

```
def calculate(a, b):  
    return a + b, a - b, a * b  
  
add, sub, mul = calculate(10, 5)  
print(add, sub, mul)
```

Output:

```
15 5 50
```

2. Variable Scope in Python

A **scope** defines the area of a program where a variable is accessible.

There are two main types:

Local Scope

Global Scope

Local Variables

Declared **inside a function**, accessible **only within that function**.

```
def my_function():  
    x = 10 # local variable  
    print("Inside function:", x)  
  
my_function()  
print("Outside function:", x) # Error
```

Output:

```
Inside function: 10  
NameError: name 'x' is not defined
```

Global Variables

Declared **outside all functions**, accessible **everywhere**.

```
x = 50 # global variable  
  
def show_value():  
    print("Inside function:", x)  
  
show_value()  
print("Outside function:", x)
```

Output:

```
Inside function: 50  
Outside function: 50
```


Quick Summary Table

Scope Type	Defined	Accessible	Example
Local	Inside a function	Only inside that function	<code>def func(): x=5</code>
Global	Outside all functions	Anywhere in the code	<code>x=10</code> (top-level variable)

Summary

- Functions are defined using `def` followed by the function name and parentheses.
- They make code modular, reusable, and easy to maintain.
- **Parameters** are variables in the function definition, and **arguments** are actual values passed.
- Three main types of arguments: **Positional**, **Keyword**, and **Default**.
- The **return** statement sends a value back to the caller.
- Variables can have **local** or **global** scope, controlling where they can be accessed.
- Use the **global** keyword to modify global variables inside functions.
-